`k8s-04` tsaks

1.Deploy an application using a Deployment with 3 replicas and a rolling update strategy.

1. Objective

The objective of this task is to create a Kubernetes Deployment with three Nginx replicas and configure the RollingUpdate deployment strategy.

2. What is a Deployment?

A Deployment is a Kubernetes resource used to manage application Pods.

It helps us to:

- Create multiple copies of an application.
- Maintain the required number of Pods.
- Update application versions.
- Roll back to previous versions.

Example: An online shopping website may run three Nginx application Pods to handle user requests.

3. What is RollingUpdate?

RollingUpdate is the default Kubernetes Deployment strategy.

It updates application Pods gradually instead of stopping all old Pods at the same time.

Example:

Suppose an application has three Pods running version 1.

When we update to version 2, Kubernetes gradually replaces the old Pods with new Pods.

This helps keep the application available during an update, depending on readiness and the configured availability settings.

4. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Container image: nginx
- Desired replicas: 3

5. Verify Kubernetes Nodes

Command:

```
None
      kubectl get nodes -o wide
```

Result:

All three nodes were in Ready status:

- k8s-master
- k8s-worker-1
- k8s-worker-2

6. Create Deployment YAML

File name: `nginx-rolling.yaml`

```
None
      apiVersion: apps/v1
      kind: Deployment
      metadata:
        name: nginx-rolling
        labels:
          app: nginx-rolling

      spec:
        replicas: 3
```

```
  selector:
    matchLabels:
      app: nginx-rolling

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1

  template:
    metadata:
      labels:
        app: nginx-rolling

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

7. Explanation of YAML

- `apiVersion: apps/v1` – API version used for Deployment.
- `kind: Deployment` – Tells Kubernetes to create a Deployment.
- `metadata.name` – Name of the Deployment.
- `replicas: 3` – Kubernetes should maintain three Pods.
- `selector.matchLabels` – Identifies the Pods managed by this Deployment.
- `template` – Defines how the Pods should be created.

- `image: nginx:latest` – Uses the Nginx container image.
- `containerPort: 80` – The Nginx container listens on port 80.
- `strategy.type: RollingUpdate` – Updates Pods gradually.
- `maxSurge: 1` – Allows one extra Pod above the desired replica count during an update.
- `maxUnavailable: 1` – Allows at most one Pod to be unavailable during an update.

8. Apply the YAML File

Command:

```
None
kubectl apply -f nginx-rolling.yaml
```

Expected output:

```
None
deployment.apps/nginx-rolling created
```

9. Verify Deployment

Command:

```
None
kubectl get deployments
```

The Deployment was created with three desired replicas.

10. Verify Pods

Command:

```
None
kubectl get pods -o wide
```

The three Pods created by this Deployment were:

Pod name	Worker node	Status
nginx-rolling-8c686d4fb-dvrdq	k8s-worker-1	Running
nginx-rolling-8c686d4fb-kw4wl	k8s-worker-1	Running
nginx-rolling-8c686d4fb-q6p9h	k8s-worker-2	Running

All three Pods were Running and Ready.

11. Verify Rollout Status

Command:

```
None
kubectl rollout status deployment/nginx-rolling
```

Output:

```
None
deployment "nginx-rolling" successfully rolled out
```

This confirms that the Deployment rollout completed successfully.

12. Verify Deployment Strategy

Command:

```
None
kubectl describe deployment nginx-rolling | grep -E
'Replicas:|StrategyType:|RollingUpdateStrategy:'
```

Output:

```
None
Replicas:                3 desired | 3 updated | 3 total
| 3 available | 0 unavailable
StrategyType:            RollingUpdate
RollingUpdateStrategy:   1 max unavailable, 1 max surge
```

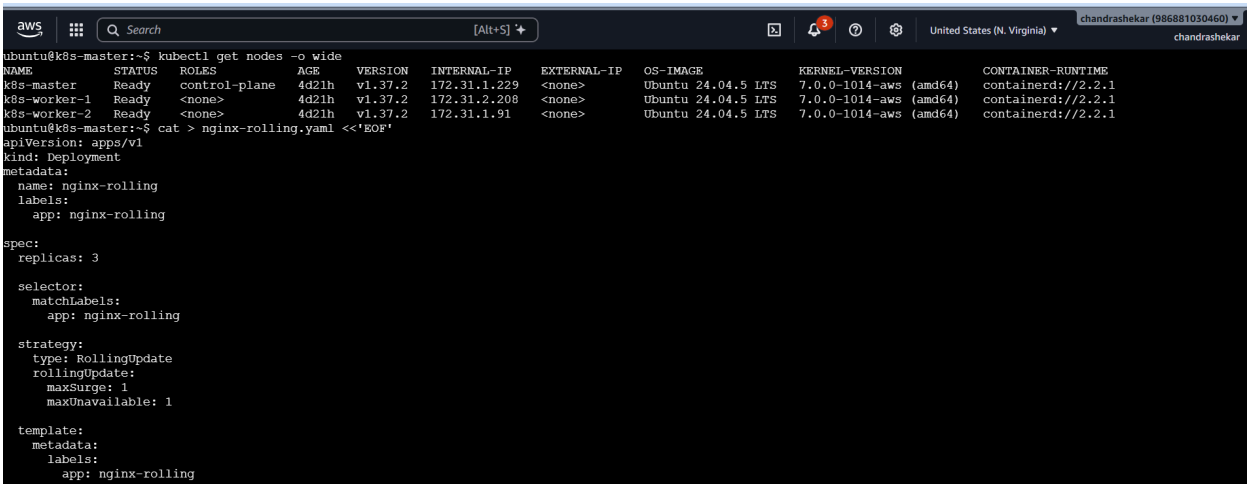
This confirms:

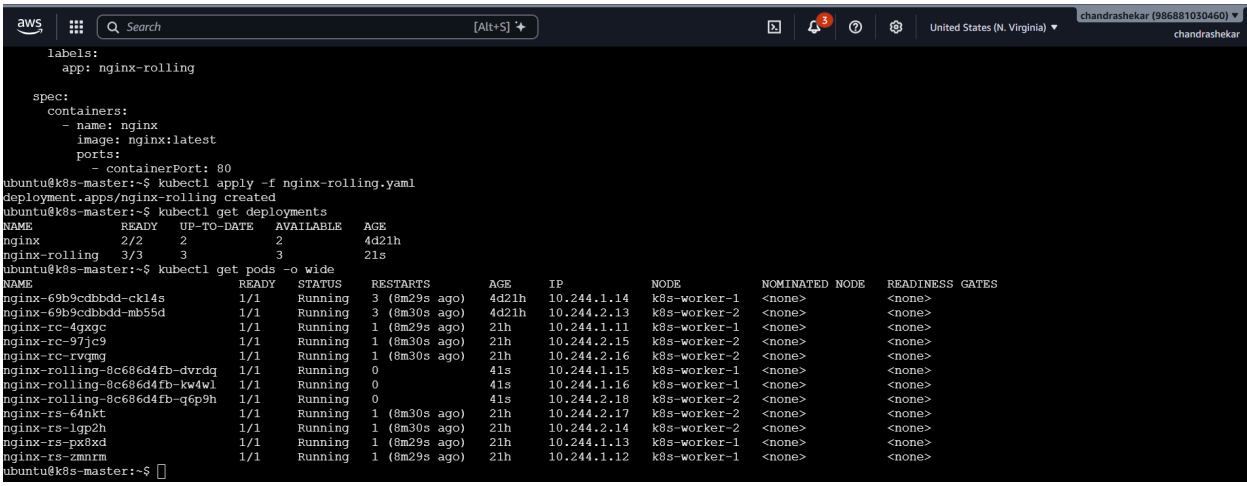
- Three replicas are available.
- RollingUpdate is configured.
- One extra Pod is allowed during an update.
- At most one Pod can be unavailable during an update.

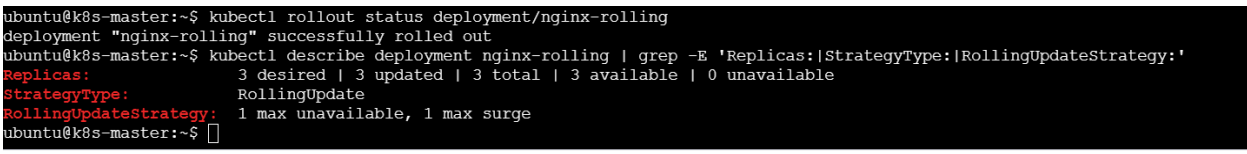
13. Final Result

Successfully created an Nginx Deployment with three replicas using the RollingUpdate strategy on the AWS EC2 Kubernetes cluster.

All three Pods were running across the two worker nodes, and the rollout status was successful.







Task 1: Completed.

2.Configure a Deployment with a Recreate strategy and observe the downtime.

1. Objective

The objective of this task is to create a Kubernetes Deployment with three replicas using the Recreate strategy, update the application image, and observe temporary downtime.

2. What is Recreate Strategy?

Recreate is a Kubernetes Deployment strategy.

When we update an application using Recreate:

1. Kubernetes deletes all existing Pods.
2. It waits for the old Pods to terminate.
3. It creates new Pods with the updated application image.
4. The application becomes available after the new Pods are ready.

During this process, there can be temporary downtime because the old Pods are deleted before the new Pods become ready.

Real-world example:

Imagine an online shopping website running three application Pods.

When the company updates the application using Recreate, all three old Pods are stopped first. The new Pods are then created with the updated application.

During this gap, customers may temporarily be unable to access the website.

3. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Initial container image: nginx
- Updated container image: httpd
- Number of replicas: 3

4. Create Deployment YAML

File name: `nginx-recreate.yaml`

```
None
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-recreate
  labels:
    app: nginx-recreate

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-recreate
```

```
  strategy:
    type: Recreate

  template:
    metadata:
      labels:
        app: nginx-recreate

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

5. Explanation of Important YAML Fields

- `kind: Deployment` – Creates a Kubernetes Deployment.
- `metadata.name` – Name of the Deployment.
- `replicas: 3` – Maintains three application Pods.
- `selector.matchLabels` – Selects Pods having the label `app: nginx-recreate`.
- `strategy.type: Recreate` – Deletes old Pods before creating new Pods.
- `template` – Contains the Pod configuration.
- `image: nginx:latest` – Initially uses the Nginx image.
- `containerPort: 80` – Application listens on port 80.

6. Apply the Deployment

Command:

```
None
kubectl apply -f nginx-recreate.yaml
```

Output:

```
None
deployment.apps/nginx-recreate created
```

7. Verify Deployment

Command:

```
None
kubectl get deployments nginx-recreate
```

Output:

```
None
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
nginx-recreate    3/3     3            3           44s
```

This confirms that all three replicas are available.

8. Verify Pods

Command:

```
None
kubectl get pods -l app=nginx-recreate -o wide
```

Initial Pods:

Pod name	Worker node	Status
nginx-recreate-75844d7d6b-d4msr	k8s-worker-2	Running

nginx-recreate-75844d7d6b-kf8vn	k8s-worker-1	Running
nginx-recreate-75844d7d6b-qmkhj	k8s-worker-1	Running

All three Pods were Running.

9. Create a ClusterIP Service

We created a Service to access the application through a stable ClusterIP address.

Command:

```
None
kubectl expose deployment nginx-recreate \
  --name=nginx-recreate-svc \
  --port=80 \
  --target-port=80 \
  --type=ClusterIP
```

Output:

```
None
service/nginx-recreate-svc exposed
```

Verify the Service:

```
None
kubectl get svc nginx-recreate-svc
```

Result:

```
None
NAME                 TYPE        CLUSTER-IP      PORT(S)
nginx-recreate-svc   ClusterIP   10.101.224.31   80/TCP
```

Verify its endpoints:

```
None
      kubectl get endpoints nginx-recreate-svc
```

The Service initially had three Pod endpoints:

- 10.244.1.17:80
- 10.244.1.18:80
- 10.244.2.19:80

10. Create a Test Client Pod

We created a BusyBox Pod to test application availability.

Command:

```
None
      kubectl run recreate-client \
        --image=busybox:1.36 \
        --restart=Never \
        --labels="tier=backend" \
        -- sleep 3600
```

Why did we add `tier=backend`?

An earlier ReplicaSet from K8S-03 used the selector `NotIn (backend)`. That selector also matches Pods without a `tier` label. The first test client, which had no `tier` label, was adopted and deleted by that ReplicaSet.

Adding `tier=backend` prevented that selector from matching our test client.

Verify:

```
None
      kubectl get pod recreate-client --show-labels
```

The test Pod was Running with label `tier=backend`.

11. Test the Initial Nginx Application

Command:

```
None
      kubectl exec recreate-client -- wget -qO-
      http://nginx-recreate-svc
```

The output displayed:

```
None
      <title>Welcome to nginx!</title>
      <h1>Welcome to nginx!</h1>
```

This confirmed that the Nginx application was working.

12. Update Nginx to Apache HTTPD

We updated the Deployment image using:

```
None
      kubectl set image deployment/nginx-recreate
      nginx=httpd:latest
```

This changed the application image from Nginx to Apache HTTPD.

Verify the new Pods:

```
None
      kubectl get pods -l app=nginx-recreate -o wide
```

New Pods were created with a different ReplicaSet hash:

New Pod name	Worker node	Status

nginx-recreate-5f44dc8fbb-574zk	k8s-worker-2	Running
nginx-recreate-5f44dc8fbb-64hmq	k8s-worker-1	Running
nginx-recreate-5f44dc8fbb-xzvqq	k8s-worker-1	Running

The different Pod names confirmed that the old Pods had been replaced.

13. Verify Apache HTTPD Application

Command:

```
None
kubectl exec recreate-client -- wget -qO-
http://nginx-recreate-svc
```

Output:

```
None
<title>It works! Apache httpd</title>
<p>It works!</p>
```

This confirmed that the updated HTTPD application was working through the same Service.

14. Observe Downtime During Recreate Update

To monitor application availability, we continuously sent HTTP requests to the Service.

Monitoring command:

```
None
kubectl exec recreate-client -- sh -c '
while true
do
```

```
  if wget -T 2 -qO- http://nginx-recreate-svc >/dev/null
2>&1
  then
    echo "$(date +%T) SUCCESS"
  else
    echo "$(date +%T) FAILED"
  fi
  sleep 1
done'
```

We then changed the image back to Nginx:

```
None
kubectl set image deployment/nginx-recreate
nginx=nginx:latest
```

During this update, the monitoring output showed:

```
None
09:47:07 SUCCESS
09:47:08 SUCCESS
09:47:09 FAILED
09:47:12 FAILED
09:47:13 SUCCESS
09:47:14 SUCCESS
```

What does this output mean?

- SUCCESS – The Service successfully returned an HTTP response.
- FAILED – The HTTP request did not succeed within the configured timeout.
- SUCCESS again – The new Pods became ready and the Service started responding again.

This demonstrated temporary application unavailability during the Recreate update.

The exact downtime duration was not measured.

15. Final Result

Successfully created a Kubernetes Deployment with three replicas using the Recreate strategy.

Updated the application from Nginx to Apache HTTPD and verified the new application response.

Changed the image back to Nginx while continuously monitoring the Service. The monitoring output showed failed requests during the replacement and successful requests after the new Pods became ready.

16. Difference Between RollingUpdate and Recreate

RollingUpdate	Recreate
Replaces Pods gradually	Deletes all old Pods first
Old and new Pods can temporarily coexist	Old Pods are removed before new Pods are created
Can maintain application availability	Causes temporary downtime during replacement
Default Deployment strategy	Must be configured explicitly

terminal-1



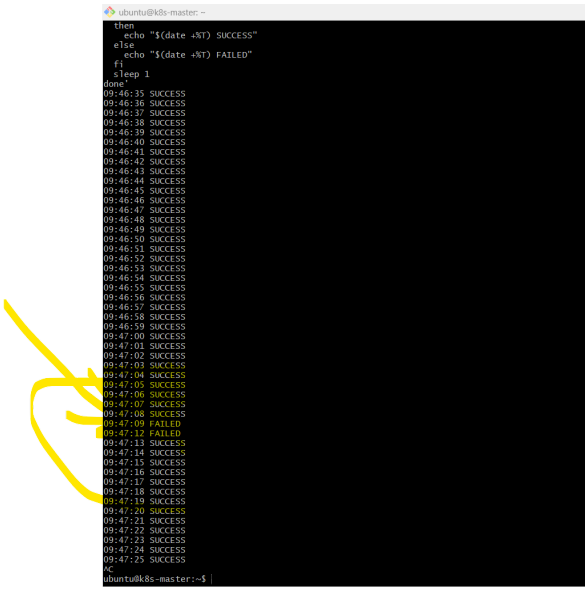

```
ubuntu@k8s-master:~$ kubectl get pod recreate-client
Error from server (NotFound): pods "recreate-client" not found
ubuntu@k8s-master:~$ kubectl exec recreate-client -- wget -qO- http://nginx-recreate-svc
Error from server (NotFound): pods "recreate-client" not found
ubuntu@k8s-master:~$ kubectl get pods -A | grep recreate
default        nginx-recreate-75844d7d6b-d4msr      1/1     Running   0             6m5s
default        nginx-recreate-75844d7d6b-kf8vn      1/1     Running   0             6m5s
default        nginx-recreate-75844d7d6b-qmkhj      1/1     Running   0             6m5s
ubuntu@k8s-master:~$ kubectl get events --sort-by=.lastTimestamp | tail -20
6m29s       Normal    SuccessfulCreate          replicaset/nginx-recreate-75844d7d6b   Created pod: nginx-recreate-75844d7d6b-kf8vn
6m29s       Normal    Scheduled                 pod/nginx-recreate-75844d7d6b-kf8vn    Successfully assigned default/nginx-recreate-75844d7d6b-kf8vn to k8s-worker-1
6m29s       Normal    Scheduled                 pod/nginx-recreate-75844d7d6b-d4msr    Successfully assigned default/nginx-recreate-75844d7d6b-d4msr to k8s-worker-2
6m29s       Normal    SuccessfulCreate          replicaset/nginx-recreate-75844d7d6b   Created pod: nginx-recreate-75844d7d6b-qmkhj
6m29s       Normal    SuccessfulCreate          replicaset/nginx-recreate-75844d7d6b   Created pod: nginx-recreate-75844d7d6b-d4msr
6m29s       Normal    Scheduled                 pod/nginx-recreate-75844d7d6b-qmkhj    Successfully assigned default/nginx-recreate-75844d7d6b-qmkhj to k8s-worker-1
6m28s       Normal    Created                   pod/nginx-recreate-75844d7d6b-d4msr    Container created
6m28s       Normal    Pulled                    pod/nginx-recreate-75844d7d6b-qmkhj    Successfully pulled image "nginx:latest" in 147ms (147ms including waiting). Image
 size: 63933865 bytes.
6m28s       Normal    Pulling                   pod/nginx-recreate-75844d7d6b-kf8vn    Pulling image "nginx:latest"
6m28s       Normal    Pulling                   pod/nginx-recreate-75844d7d6b-d4msr    Pulling image "nginx:latest"
6m28s       Normal    Created                   pod/nginx-recreate-75844d7d6b-kf8vn    Container created
6m28s       Normal    Pulled                    pod/nginx-recreate-75844d7d6b-kf8vn    Successfully pulled image "nginx:latest" in 136ms (269ms including waiting). Image
 size: 63933865 bytes.
6m28s       Normal    Started                   pod/nginx-recreate-75844d7d6b-d4msr    Container started
6m28s       Normal    Started                   pod/nginx-recreate-75844d7d6b-kf8vn    Container started
6m28s       Normal    Pulled                    pod/nginx-recreate-75844d7d6b-d4msr    Successfully pulled image "nginx:latest" in 125ms (125ms including waiting). Image
 size: 63933865 bytes.
6m28s       Normal    Pulling                   pod/nginx-recreate-75844d7d6b-qmkhj    Pulling image "nginx:latest"
```

```
6m28s       Normal    Pulling                   pod/nginx-recreate-75844d7d6b-qmkhj    Pulling image "nginx:latest"
6m28s       Normal    Started                   pod/nginx-recreate-75844d7d6b-qmkhj    Container started
6m28s       Normal    Created                   pod/nginx-recreate-75844d7d6b-qmkhj    Container created
115s        Normal    SuccessfulDelete          replicaset/nginx-rs-notin              Deleted pod: recreate-client
115s        Normal    Scheduled                 pod/recreate-client                    Successfully assigned default/recreate-client to k8s-worker-2
ubuntu@k8s-master:~$ kubectl run recreate-client \
  --image=busybox:1.36 \
  --restart=Never \
  --labels="tier=backend" \
  -- sleep 3600
pod/recreate-client created
ubuntu@k8s-master:~$ kubectl get pod recreate-client --show-labels
NAME              READY   STATUS    RESTARTS   AGE   LABELS
recreate-client   1/1     Running   0          15s   tier=backend
ubuntu@k8s-master:~$ kubectl exec recreate-client -- wget -qO- http://nginx-recreate-svc
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>
```

```
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@k8s-master:~$
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-recreate nginx=httpd:latest
deployment.apps/nginx-recreate image updated
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-recreate -o wide
NAME                              READY   STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   READINESS GATES
nginx-recreate-5f44dc8fbb-574zk   1/1     Running   0          68s   10.244.2.21   k8s-worker-2   <none>           <none>
nginx-recreate-5f44dc8fbb-64hmq   1/1     Running   0          67s   10.244.1.20   k8s-worker-1   <none>           <none>
nginx-recreate-5f44dc8fbb-xzvqq   1/1     Running   0          68s   10.244.1.19   k8s-worker-1   <none>           <none>
ubuntu@k8s-master:~$ kubectl exec recreate-client -- wget -qO- http://nginx-recreate-svc
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-recreate nginx=nginx:latest
deployment.apps/nginx-recreate image updated
ubuntu@k8s-master:~$
```

Terminal-2



Task 2: Completed.

3.Update an existing Deployment and perform a rollback to the previous version

1. Objective

The objective of this task is to update an existing Kubernetes Deployment with a new application image and roll back to the previous version.

In this practical, we:

- Created an Nginx Deployment with 3 replicas.
- Updated the application image from Nginx to Apache HTTPD.
- Checked the Deployment rollout status and revision history.
- Performed a rollback to the previous Nginx version.
- Verified that the rollback was successful.

2. What is Rollout?

A rollout means releasing or applying a new version of an application.

Example:

A company is running an application using Nginx. Now the company wants to update its application to a new version.

Kubernetes Deployment manages the update process.

3. What is Revision?

A revision is a record of a Deployment's Pod template configuration.

Kubernetes maintains Deployment revisions so that we can return to a previous application version when required.

Example:

- Revision 1 – Original Nginx image.
- Revision 2 – Updated HTTPD image.
- Revision 3 – Nginx image restored using rollback.

4. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Deployment name: nginx-rollback
- Initial image: nginx
- Updated image: httpd
- Number of replicas: 3

5. Create Deployment YAML

File name: `nginx-rollback.yaml`

None

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-rollback
  labels:
    app: nginx-rollback

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-rollback

  template:
    metadata:
      labels:
        app: nginx-rollback

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Explanation of important fields

- `apiVersion: apps/v1` – API version for Deployment.
- `kind: Deployment` – Creates a Deployment resource.

- `metadata.name` – Name of the Deployment.
- `replicas: 3` – Maintains three Pods.
- `selector.matchLabels` – Identifies the Pods managed by the Deployment.
- `template` – Defines the Pod configuration.
- `image: nginx:latest` – Initially uses the Nginx container image.
- `containerPort: 80` – Application listens on port 80.

6. Apply the Deployment

Command:

```
None
kubectl apply -f nginx-rollback.yaml
```

Output:

```
None
deployment.apps/nginx-rollback created
```

7. Verify the Deployment

Command:

```
None
kubectl get deployments nginx-rollback
```

Output:

```
None
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
nginx-rollback   3/3     3            3           15s
```

This confirms that all three replicas were created and available.

8. Verify Pods

Command:

```
None
      kubectl get pods -l app=nginx-rollback -o wide
```

Initial Pods:

Pod	Worker node
nginx-rollback-7b985d8585-5cp8k	worker-1
nginx-rollback-7b985d8585-sm9pv	worker-1
nginx-rollback-7b985d8585-wfst9	worker-2

All three Pods were Running.

9. Check Initial Rollout Status

Command:

```
None
kubectl rollout status deployment/nginx-rollback
```

Output:

```
None
deployment "nginx-rollback" successfully rolled out
```

This confirms that the initial Deployment completed successfully.

10. Check Initial Revision History

Command:

```
None
kubectl rollout history deployment/nginx-rollback
```

Output:

```
None
deployment.apps/nginx-rollback
REVISION  CHANGE-CAUSE
1         <none>
```

Revision 1 represents our original Nginx Deployment.

The change-cause is shown as `<none>` because we did not configure a change-cause annotation.

11. Update Nginx to Apache HTTPD

Now we updated the container image from Nginx to Apache HTTPD.

Command:

```
None
      kubectl set image deployment/nginx-rollback
      nginx=httpd:latest
```

Output:

```
None
      deployment.apps/nginx-rollback image updated
```

What happened?

The Deployment's Pod template was changed.

Kubernetes created a new ReplicaSet and replaced the old Nginx Pods with new HTTPD Pods.

12. Verify Updated Deployment

Command:

```
None
      kubectl rollout status deployment/nginx-rollback
```

Output:

```
None
      deployment "nginx-rollback" successfully rolled out
```

This confirms that the HTTPD update completed successfully.

Check the Pods:

```
None
      kubectl get pods -l app=nginx-rollback -o wide
```

Updated Pods:

Pod	Worker node	Status
nginx-rollback-848755477c-hrhv9	worker-2	Running
nginx-rollback-848755477c-vd8cd	worker-1	Running
nginx-rollback-848755477c-wktjp	worker-1	Running

The new Pods have different names because Kubernetes created them from the updated Pod template.

13. Verify Revision History After Update

Command:

```
None
kubectl rollout history deployment/nginx-rollback
```

Output:

```
None
deployment.apps/nginx-rollback
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

Now we have two revisions:

- Revision 1 – Nginx.
- Revision 2 – HTTPD.

14. Perform Rollback to Revision 1

Scenario

Imagine that after updating the application to HTTPD, we find an application problem.

We want to return to the previous working Nginx version.

This process is called **Rollback**.

Command:

```
None
      kubectl rollout undo deployment/nginx-rollback
      --to-revision=1
```

Output:

```
None
      deployment.apps/nginx-rollback rolled back
```

Kubernetes restored the Pod template configuration from Revision 1.

Note: Kubernetes displayed a warning about the `last-applied-configuration` annotation because the Deployment was originally created using `kubectl apply`. The rollback still succeeded. This warning does not mean the rollback failed.

15. Verify Rollback Status

Command:

```
None
      kubectl rollout status deployment/nginx-rollback
```

Output:

```
None
      deployment "nginx-rollback" successfully rolled out
```

This confirms that rollback completed successfully.

16. Verify the Current Application Image

Command:

```
None
      kubectl get deployment nginx-rollback -o
      jsonpath='{.spec.template.spec.containers[0].image}{"\n"
      }'
```

Output:

```
None
      nginx:latest
```

This confirms that the application is back to the original Nginx image.

17. Check Revision History After Rollback

Command:

```
None
      kubectl rollout history deployment/nginx-rollback
```

Output:

```
None
      deployment.apps/nginx-rollback
      REVISION  CHANGE-CAUSE
      2         <none>
      3         <none>
```

Why is Revision 3 created?

Rollback does not simply erase the newer revision.

Kubernetes creates a new revision containing the restored configuration.

Our revision history is therefore:

Revision	Application image	Meaning
Revision 1	nginx	Original version
Revision 2	httpd	Updated version
Revision 3	nginx	Restored through rollback

Revision 1's history entry is no longer shown because its configuration was rolled back and its revision number was moved to the new revision.

18. Important Commands for Revision and Rollback

Check rollout status:

```
None
kubectl rollout status deployment/nginx-rollback
```

Check rollout history:

```
None
kubectl rollout history deployment/nginx-rollback
```

Update application image:

```
None
kubectl set image deployment/nginx-rollback
nginx=httpd:latest
```

Roll back to the previous revision:

```
None
kubectl rollout undo deployment/nginx-rollback
```

Roll back to a specific revision:

```
None
      kubectl rollout undo deployment/nginx-rollback
      --to-revision=1
```

Check the current image:

```
None
      kubectl get deployment nginx-rollback -o
      jsonpath='{.spec.template.spec.containers[0].image}{"\n"
      }'
```

19. Final Result

Successfully created a Kubernetes Deployment with three Nginx replicas.

Updated the application image from Nginx to Apache HTTPD and verified that the new rollout completed.

Performed a rollback to Revision 1 and verified that the Deployment returned to `nginx:latest`.

20. Interview Questions

Q1. What is a Kubernetes Deployment?

A Deployment manages application Pods and helps us update, scale and roll back applications.

Q2. What is a rollout?

A rollout is the process of releasing a new version or applying changes to a Deployment.

Q3. What is a revision?

A revision stores the Deployment's Pod template configuration for a particular version.

Q4. What is rollback?

Rollback means returning an application to a previous Deployment version.

Q5. Which command is used to check Deployment history?

```
None
      kubectl rollout history deployment/DEPLOYMENT_NAME
```

Q6. Which command is used to roll back to a specific revision?

```
None
      kubectl rollout undo deployment/DEPLOYMENT_NAME
      --to-revision=1
```

Q7. Does rollback delete the complete Deployment?

No. It restores the previous Pod template configuration and the Deployment continues managing its Pods.

Q8. Why was Revision 3 created after rollback?

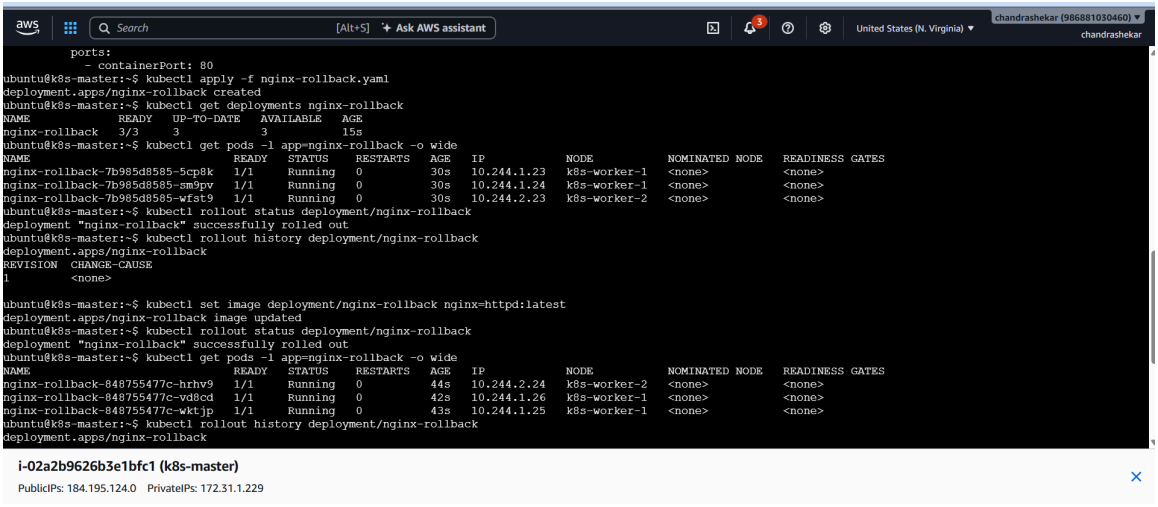
Because Kubernetes records the restored configuration as a new Deployment revision.

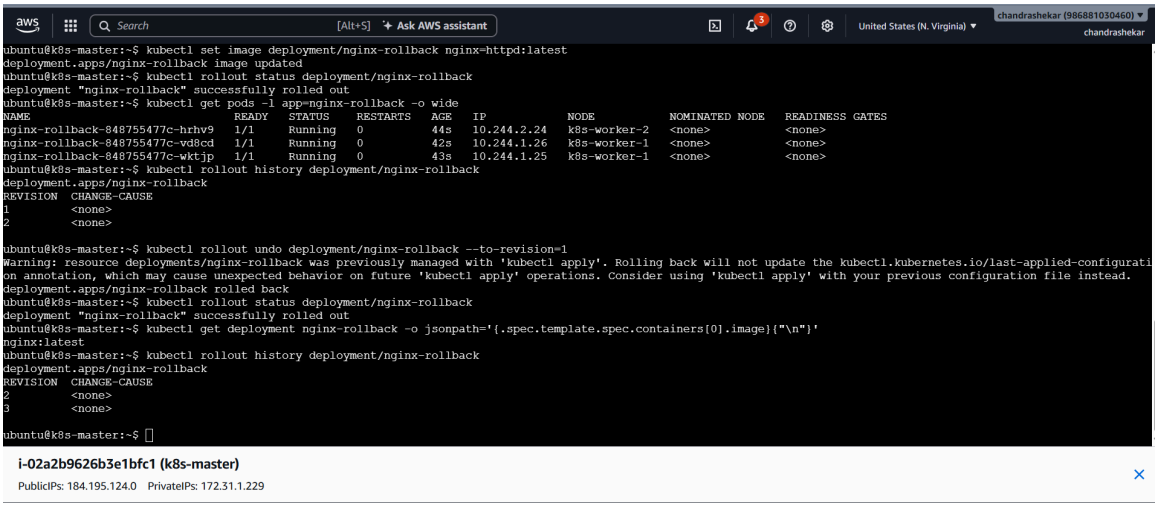
Q9. What is the difference between rollout and rollback?

Rollout applies a new Deployment version. Rollback returns to a previously stored Deployment version.

Q10. How do you verify the current container image?

```
None
      kubectl get deployment nginx-rollback -o
      jsonpath='{.spec.template.spec.containers[0].image}{"\n"
      }'
```





K8S-04 Task 3: Completed successfully.

4.Modify a Deployment to add resource requests and limits for CPU and memory.

1. Objective

The objective of this task is to modify an existing Kubernetes Deployment by adding CPU and memory resource requests and limits to its containers.

In this practical, we:

- Modified the existing `nginx-rollback` Deployment.
- Configured CPU and memory requests.
- Configured CPU and memory limits.
- Applied the updated YAML file.

- Verified the resource settings using `kubectl describe pod`.
- Checked the Pod QoS Class.

2. What are Resource Requests?

A resource request tells Kubernetes how much CPU and memory a container needs.

The Kubernetes scheduler uses these requested values to decide which worker node has enough available resources to run the Pod.

Real-world example:

Imagine a company has an application that needs at least 100m CPU and 128Mi memory.

We configure these values as resource requests. Kubernetes looks for a worker node that has enough available resources to schedule the Pod.

3. What are Resource Limits?

A resource limit specifies the maximum CPU and memory a container is allowed to use.

Real-world example:

Imagine an application normally needs 100m CPU and 128Mi memory, but sometimes needs more resources.

We can configure limits of 500m CPU and 256Mi memory.

The container can use more than its request when resources are available, but it cannot exceed its configured limits.

4. Difference Between Requests and Limits

Resource	Request	Limit
CPU	100m	500m
Memory	128Mi	256Mi

- Request = Resources Kubernetes considers when scheduling the Pod.

- Limit = Maximum resource usage allowed for the container.

5. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Deployment name: nginx-rollback
- Container image: nginx
- Number of replicas: 3

6. Update Deployment YAML

File name: `nginx-rollback.yaml`

We modified the existing Deployment YAML and added the `resources` section inside the container specification.

```
None
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-rollback
  labels:
    app: nginx-rollback

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-rollback

  template:
```

```
    metadata:
      labels:
        app: nginx-rollback

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

7. Explanation of Resource YAML

Requests

```
None
  requests:
    cpu: "100m"
    memory: "128Mi"
```

- `cpu: 100m` – Requests 100 millicores, equal to 0.1 CPU core.
- `memory: 128Mi` – Requests 128 mebibytes of memory.

Limits

```
None
      limits:
        cpu: "500m"
        memory: "256Mi"
```

- `cpu: 500m` – Maximum CPU usage is 0.5 CPU core.
- `memory: 256Mi` – Maximum memory usage is 256 mebibytes.

These values are configured per container, not for the entire Deployment.

Since the Deployment has three replicas, each Pod has the same resource configuration.

8. Apply the Updated Deployment

Command:

```
None
      kubectl apply -f nginx-rollback.yaml
```

Output:

```
None
      deployment.apps/nginx-rollback configured
```

This means Kubernetes updated the existing Deployment with the new resource configuration.

9. Verify Rollout Status

Command:

```
None
      kubectl rollout status deployment/nginx-rollback
```

Output:

```
None
      deployment "nginx-rollback" successfully rolled out
```

This confirms that the updated Deployment completed successfully.

10. Verify Deployment Replicas

Command:

```
None
      kubectl get deployments nginx-rollback
```

Output:

```
None
      NAME             READY   UP-TO-DATE   AVAILABLE   AGE
      nginx-rollback   3/3     3            3           18m
```

This confirms:

- 3 desired replicas.
- 3 updated replicas.
- 3 available replicas.

11. Verify CPU and Memory Requests and Limits

Command:

```
None
      kubectl describe pod nginx-rollback
```

The output displayed the resource configuration for all three matching Pods.

Example from the output:

```
None
      Containers:
        nginx:
          Image: nginx:latest

          Limits:
            cpu:     500m
            memory:  256Mi

          Requests:
            cpu:        100m
            memory:     128Mi

          State: Running
          Ready: True
```

This confirms that Kubernetes successfully applied the CPU and memory requests and limits.

All three Pods showed the same resource settings.

12. Understand QoS Class – Burstable

In the Pod description, we observed:

```
None
      QoS Class: Burstable
```

Kubernetes assigns a Quality of Service (QoS) Class to Pods based on their resource requests and limits.

Three main QoS Classes are:

QoS Class	Simple explanation

Guaranteed	Every container has equal CPU and memory requests and limits, with both specified
Burstable	At least one resource request or limit is specified, and the Pod does not meet Guaranteed conditions
BestEffort	No CPU or memory requests or limits are specified

In our practical, requests are lower than limits, so the Pod belongs to the Burstable class.

13. Important Concept – CPU and Memory Units

CPU

Value	Meaning
100m	0.1 CPU core
250m	0.25 CPU core
500m	0.5 CPU core
1000m	1 CPU core

Memory

Value	Meaning
128Mi	128 mebibytes
256Mi	256 mebibytes
512Mi	512 mebibytes

1Gi	1 gibibyte

14. Important Commands Used

Apply the updated YAML:

```
None
kubectl apply -f nginx-rollback.yaml
```

Check Deployment status:

```
None
kubectl rollout status deployment/nginx-rollback
```

Check replicas:

```
None
kubectl get deployments nginx-rollback
```

Get Pod details:

```
None
kubectl get pods -l app=nginx-rollback -o wide
```

Describe Pods and check resource configuration:

```
None
kubectl describe pod nginx-rollback
```

15. Final Result

Successfully modified the existing Nginx Deployment by configuring:

- CPU request: 100m
- CPU limit: 500m
- Memory request: 128Mi

- Memory limit: 256Mi

Applied the updated YAML successfully and verified that all three Pods were Running and Ready.

The Pod description confirmed the configured requests and limits, and the QoS Class was Burstable.

16. Interview Questions and Answers

Q1. What is a resource request in Kubernetes?

A resource request specifies the CPU and memory needed by a container. The Kubernetes scheduler considers these values when selecting a node.

Q2. What is a resource limit?

A resource limit specifies the maximum CPU and memory a container can use.

Q3. What is the difference between requests and limits?

Requests are used for scheduling. Limits restrict resource usage.

Q4. What does 100m CPU mean?

It means 100 millicores, or 0.1 CPU core.

Q5. What happens if a Pod requests more CPU or memory than any node has available?

The scheduler cannot place the Pod, so it can remain in Pending status.

Q6. What happens when a container exceeds its memory limit?

It can be terminated with an Out Of Memory (OOM) kill.

Q7. What happens when a container tries to exceed its CPU limit?

CPU usage is throttled. The container is not normally terminated just because it reaches its CPU limit.

Q8. What is QoS Class?

QoS means Quality of Service. Kubernetes uses it to classify Pods according to their CPU and memory resource configuration.

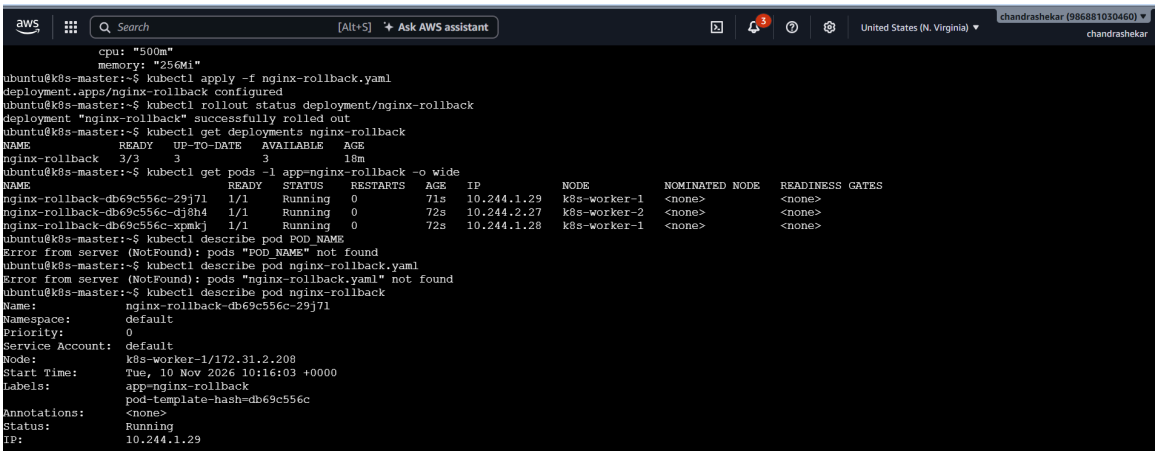
Q9. What are the three QoS Classes?

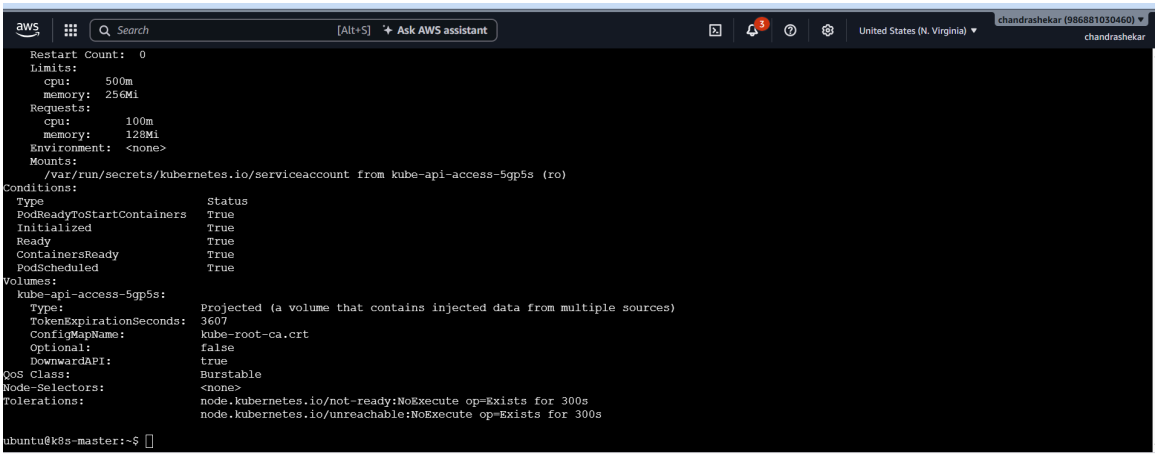
Guaranteed, Burstable and BestEffort.

Q10. Which command helps us check resource requests and limits?

None

```
kubectl describe pod POD_NAME
```





K8S-04 Task 4: Completed successfully.

5.Create a Deployment with MaxSurge and MaxUnavailable configurations.

K8S-04 – Task 5: Configure MaxSurge and MaxUnavailable in Deployment

1. Objective

The objective of this task is to create a Kubernetes Deployment with three replicas and configure MaxSurge and MaxUnavailable values to control how Pods are replaced during a RollingUpdate.

In this practical, we:

- Created an Nginx Deployment with three replicas.
- Configured MaxSurge as 2.
- Configured MaxUnavailable as 1.
- Updated the application image from Nginx to HTTPD.
- Observed Pod creation and deletion during the update.
- Verified the final Deployment and Pods.

2. What is MaxSurge?

MaxSurge specifies how many extra Pods Kubernetes is allowed to create above the desired number of replicas during a RollingUpdate.

Example:

If a Deployment has:

- Replicas = 3
- MaxSurge = 2

Kubernetes can temporarily run up to 5 Pods during the update.

Calculation:

3 desired Pods + 2 extra Pods = 5 maximum Pods.

3. What is MaxUnavailable?

MaxUnavailable specifies the maximum number of Pods that can be unavailable during a RollingUpdate.

Example:

If a Deployment has:

- Replicas = 3
- MaxUnavailable = 1

At most one Pod can be unavailable during the update. At least two Pods must remain available.

4. Understand Both Together

Our configuration:

Setting	Meaning
Replicas	Maintain 3 Pods
MaxSurge	Allow up to 2 extra Pods
MaxUnavailable	Allow at most 1 unavailable Pod

These two settings help Kubernetes control the replacement process during application updates.

MaxSurge controls extra Pods, while MaxUnavailable controls unavailable Pods.

5. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Deployment name: nginx-surge

- Initial image: nginx
- Updated image: httpd
- Desired replicas: 3

6. Create Deployment YAML

File name: `nginx-surge.yaml`

None

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-surge
  labels:
    app: nginx-surge

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-surge

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 1

  template:
    metadata:
```

```
      labels:
        app: nginx-surge

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

7. Explanation of Important YAML Fields

- `kind: Deployment` – Creates a Kubernetes Deployment.
- `replicas: 3` – Maintains three Pods.
- `selector.matchLabels` – Identifies the Pods managed by the Deployment.
- `strategy.type: RollingUpdate` – Updates Pods gradually.
- `maxSurge: 2` – Allows up to two extra Pods during the update.
- `maxUnavailable: 1` – Allows at most one unavailable Pod.
- `image: nginx:latest` – Initially uses the Nginx image.
- `containerPort: 80` – Application listens on port 80.

8. Apply Deployment YAML

Command:

```
None
kubectl apply -f nginx-surge.yaml
```

Output:

```
None
deployment.apps/nginx-surge created
```

9. Verify Deployment

Command:

```
None
      kubectl get deployment nginx-surge
```

Output:

```
None
      NAME          READY   UP-TO-DATE   AVAILABLE   AGE
      nginx-surge   3/3     3            3           21s
```

This confirms that all three replicas were available.

10. Verify Initial Pods

Command:

```
None
      kubectl get pods -l app=nginx-surge -o wide
```

Initial Pods:

Pod	Worker node	Status
nginx-surge-64bf686b85-49gkh	worker-1	Runni

		n g
nginx-surge-64bf686b8 5-zc6rl	worke r-2	R u n n i n g
nginx-surge-64bf686b8 5-znpsd	worke r-2	R u n n i n g

All three Pods were Running.

11. Monitor Pods During RollingUpdate

Command:

```
None
kubectl get pods -l app=nginx-surge -w
```

The `-w` option means watch.

It continuously displays changes in Pod status.

During the update, we observed:

- New Pods were created.
- New Pods moved from Pending to ContainerCreating.
- New Pods became Running.
- Old Pods moved to Terminating and then Completed.
- The updated Pods became available.

This demonstrated how Kubernetes replaces old Pods with new Pods during a RollingUpdate.

12. Update Nginx to HTTPD

We updated the container image from Nginx to Apache HTTPD.

Command:

```
None
kubectl set image deployment/nginx-surge
nginx=httpd:latest
```

This changed the Deployment's Pod template and triggered a new rollout.

13. Verify Rollout Status

Command:

```
None
kubectl rollout status deployment/nginx-surge
```

Output:

```
None
deployment "nginx-surge" successfully rolled out
```

This confirms that the application update completed successfully.

14. Verify Deployment Strategy and Replicas

Command:

```
None
kubectl describe deployment nginx-surge | grep -E
'Replicas:|StrategyType:|RollingUpdateStrategy:'
```

Output:

```
None
      Replicas:               3 desired | 3 updated | 3 total
      | 3 available | 0 unavailable
      StrategyType:           RollingUpdate
      RollingUpdateStrategy:  1 max unavailable, 2 max surge
```

This confirms that the configured RollingUpdate values were applied successfully.

15. Verify Updated Application Image

Command:

```
None
      kubectl get deployment nginx-surge -o
      jsonpath='{.spec.template.spec.containers[0].image}{"\n"
      }'
```

Output:

```
None
      httpd:latest
```

This confirms that the current application image is HTTPD.

16. Verify Final Pods

Command:

```
None
      kubectl get pods -l app=nginx-surge -o wide
```

Final Pods:

Pod	Worker node	Status
nginx-surge-5fc74fd689-82kjz	worker-1	Running
nginx-surge-5fc74fd689-k7kl8	worker-2	Running
nginx-surge-5fc74fd689-wd5vz	worker-1	Running

All three updated Pods were Running and Ready.

17. Final Result

Successfully created a Kubernetes Deployment with three replicas using RollingUpdate.

Configured:

- MaxSurge = 2
- MaxUnavailable = 1

Updated the application image from Nginx to HTTPD and observed how Kubernetes created new Pods and terminated old Pods during the rollout.

Finally, verified that all three updated Pods were running and available.

18. Interview Questions

Q1. What is MaxSurge?

It specifies how many extra Pods can be created above the desired replica count during a RollingUpdate.

Q2. What is MaxUnavailable?

It specifies the maximum number of Pods that can be unavailable during a RollingUpdate.

Q3. What happens if replicas are 3 and MaxSurge is 2?

Kubernetes can temporarily run up to 5 Pods during an update.

Q4. What happens if replicas are 3 and MaxUnavailable is 1?

At most one Pod can be unavailable during the update.

Q5. What is the purpose of RollingUpdate?

It updates application Pods gradually rather than deleting all old Pods at once.

Q6. Which command continuously watches Pods?

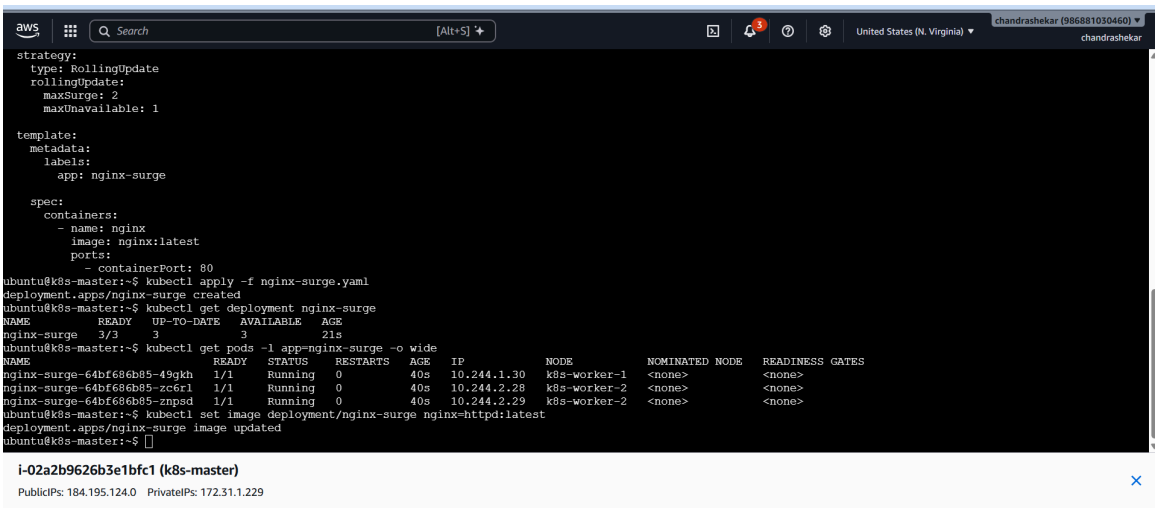
```
None
      kubectl get pods -w
```

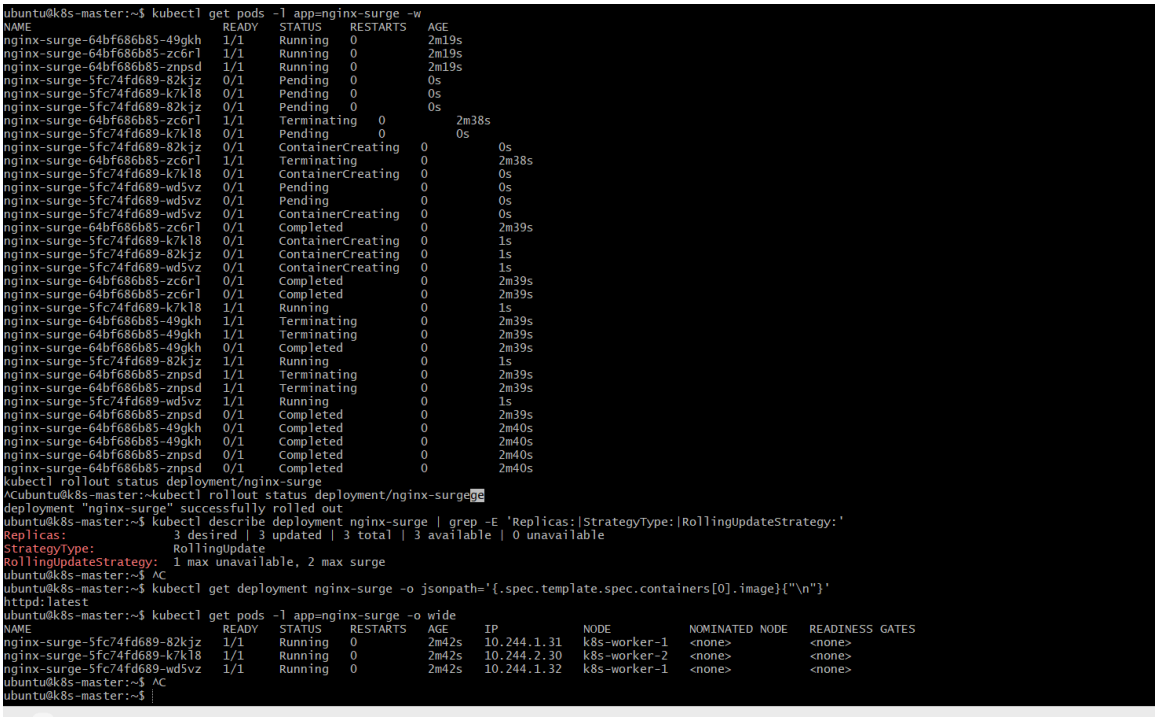
Q7. Which command checks rollout status?

```
None
      kubectl rollout status deployment/DEPLOYMENT_NAME
```

Q8. Can MaxSurge and MaxUnavailable be configured together?

Yes. Both control how Kubernetes manages Pod availability and extra Pods during a RollingUpdate.





K8S-04 Task 5: Completed successfully.

6.Set up a Deployment with a custom revision history limit.

1. Objective

The objective of this task is to create a Kubernetes Deployment with a custom revision history limit and observe how Kubernetes maintains previous Deployment versions.

In this practical, we:

- Created a Deployment with two replicas.
- Configured `revisionHistoryLimit: 2`.
- Created multiple Deployment revisions by changing environment variables.
- Checked rollout status and revision history.
- Verified how Kubernetes removed the oldest ReplicaSet.

2. What is Revision History?

Whenever we change the Pod template of a Deployment, Kubernetes creates a new revision.

A revision represents a version of the Deployment's Pod template configuration.

Example:

Revision	Application configuration
Revision 1	Initial configuration
Revision 2	Updated configuration
Revision 3	Another update
Revision 4	Latest update

Kubernetes maintains previous ReplicaSets so that we can roll back to an earlier version.

3. What is revisionHistoryLimit?

`revisionHistoryLimit` specifies how many old ReplicaSets Kubernetes should retain for rollback.

Example:

```
None
      revisionHistoryLimit: 2
```

This means:

- Kubernetes retains up to two old ReplicaSets.
- The current ReplicaSet is maintained separately.
- Older ReplicaSets beyond the configured limit can be deleted.

So, normally, we can see up to three ReplicaSets: two old ReplicaSets and one current ReplicaSet.

4. Real-World Example

Imagine a company updates its application several times.

- Revision 1 – Initial application.
- Revision 2 – First update.
- Revision 3 – Second update.
- Revision 4 – Latest update.

If the company configures `revisionHistoryLimit: 2`, Kubernetes keeps the latest two old ReplicaSets along with the current ReplicaSet.

This helps control the number of old ReplicaSets retained for rollback.

5. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Deployment name: nginx-history
- Container image: nginx
- Number of replicas: 2
- Revision history limit: 2

6. Create Deployment YAML

File name: `nginx-history.yaml`

None

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-history
  labels:
    app: nginx-history

spec:
  replicas: 2
  revisionHistoryLimit: 2

  selector:
    matchLabels:
      app: nginx-history

  template:
    metadata:
      labels:
        app: nginx-history

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

7. Explanation of Important YAML Fields

- `apiVersion: apps/v1` – API version for Deployment.
- `kind: Deployment` – Creates a Deployment resource.
- `metadata.name` – Name of the Deployment.
- `replicas: 2` – Maintains two application Pods.
- `revisionHistoryLimit: 2` – Retains up to two old ReplicaSets for rollback.
- `selector.matchLabels` – Identifies the Pods managed by the Deployment.
- `template` – Contains the Pod configuration.
- `image: nginx:latest` – Uses the Nginx container image.
- `containerPort: 80` – Application listens on port 80.

8. Apply the Deployment

Command:

```
None
kubectl apply -f nginx-history.yaml
```

Output:

```
None
deployment.apps/nginx-history created
```

This confirms that the Deployment was created successfully.

9. Verify the Deployment

Command:

```
None
kubectl get deployment nginx-history
```

Output:

```
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
nginx-history   2/2     2            2           11s
```

This confirms that:

- Two replicas were desired.
- Two replicas were updated.
- Two replicas were available.

10. Check Initial ReplicaSet

Command:

```
kubectl get rs -l app=nginx-history
```

Output:

```
NAME                       DESIRED   CURRENT   READY   AGE
nginx-history-7f6669d7d7   2         2         2       30s
```

Initially, only one ReplicaSet was created because we had only one Deployment version.

11. Create Revision 2

We changed the Deployment's Pod template by adding an environment variable.

Command:

```
kubectl set env deployment/nginx-history APP_RELEASE=v2
```

Output:

```
None
      deployment.apps/nginx-history env updated
```

Why did this create a new revision?

Because changing an environment variable inside the container specification changes the Pod template.

Kubernetes creates a new ReplicaSet and starts the updated Pods.

12. Verify Revision 2

Check rollout status:

```
None
      kubectl rollout status deployment/nginx-history
```

Output:

```
None
      deployment "nginx-history" successfully rolled out
```

Check revision history:

```
None
      kubectl rollout history deployment/nginx-history
```

Output:

```
None
      REVISION  CHANGE-CAUSE
      1         <none>
      2         <none>
```

Check ReplicaSets:

```
None
kubectl get rs -l app=nginx-history
```

Output:

```
None
NAME                       DESIRED   CURRENT   READY
AGE
nginx-history-585b69b7f8   2         2         2
43s
nginx-history-7f6669d7d7   0         0         0
2m35s
```

Explanation:

- The new ReplicaSet has two Pods.
- The old ReplicaSet has zero Pods.
- The old ReplicaSet is retained for rollback.

13. Create Revision 3

We changed the environment variable again.

Command:

```
None
kubectl set env deployment/nginx-history APP_RELEASE=v3
```

Output:

```
None
deployment.apps/nginx-history env updated
```

Check rollout status:

```
None
      kubectl rollout status deployment/nginx-history
```

Output:

```
None
      deployment "nginx-history" successfully rolled out
```

Check revision history:

```
None
      kubectl rollout history deployment/nginx-history
```

Output:

```
None
      REVISION  CHANGE-CAUSE
      1         <none>
      2         <none>
      3         <none>
```

Check ReplicaSets:

```
None
      kubectl get rs -l app=nginx-history
```

Output:

```
None
      NAME                       DESIRED   CURRENT   READY
      AGE
      nginx-history-585b69b7f8   0         0         0
      2m34s
```

```
nginx-history-7f6669d7d7   0         0         0
4m26s
nginx-history-86f475fcd9   2         2         2
38s
```

Explanation:

- Revision 1 ReplicaSet has zero Pods.
- Revision 2 ReplicaSet has zero Pods.
- Revision 3 ReplicaSet has two running Pods.

At this stage, the current ReplicaSet and two old ReplicaSets are retained.

14. Create Revision 4

We changed the environment variable once more.

Command:

```
None
kubectl set env deployment/nginx-history APP_RELEASE=v4
```

Output:

```
None
deployment.apps/nginx-history env updated
```

Check rollout status:

```
None
kubectl rollout status deployment/nginx-history
```

Output:

```
None
deployment "nginx-history" successfully rolled out
```

Check revision history:

```
None
kubectl rollout history deployment/nginx-history
```

Output:

```
None
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
4         <none>
```

Notice that Revision 1 is no longer available in the retained history.

15. Verify ReplicaSets After Revision 4

Command:

```
None
kubectl get rs -l app=nginx-history
```

Output:

```
None
NAME                       DESIRED   CURRENT   READY
AGE
nginx-history-585b69b7f8   0         0         0
4m32s
nginx-history-66b76ccf76   2         2         2
28s
nginx-history-86f475fcd9   0         0         0
2m36s
```

Explanation:

- Two old ReplicaSets have zero Pods and are retained for rollback.
- The current ReplicaSet has two running Pods.
- The oldest ReplicaSet was removed because the revision history limit is 2.

16. Understand the Final Revision History

Our practical created four revisions.

Revision	Status
Revision 1	Removed from retained history
Revision 2	Old ReplicaSet retained
Revision 3	Old ReplicaSet retained
Revision 4	Current active revision

This demonstrates that `revisionHistoryLimit: 2` retains up to two old ReplicaSets in addition to the current ReplicaSet.

17. Important Commands Used

Apply Deployment:

```
None
kubectl apply -f nginx-history.yaml
```

Check Deployment:

```
None
kubectl get deployment nginx-history
```

Check ReplicaSets:

```
None
kubectl get rs -l app=nginx-history
```

Update environment variable and create a new revision:

```
None
kubectl set env deployment/nginx-history APP_RELEASE=v2
```

Check rollout status:

```
None
kubectl rollout status deployment/nginx-history
```

Check revision history:

```
None
kubectl rollout history deployment/nginx-history
```

18. Final Result

Successfully created a Kubernetes Deployment with two replicas and configured:

```
None
revisionHistoryLimit: 2
```

Created four Deployment revisions by changing the environment variable from v2 to v3 and v4 after the initial configuration.

Verified that Kubernetes retained two old ReplicaSets and the current ReplicaSet. The oldest revision was removed from the retained history.

19. Interview Questions and Answers

Q1. What is revisionHistoryLimit?

It specifies how many old ReplicaSets Kubernetes should retain for rollback.

Q2. What is the default revision history limit?

The Kubernetes Deployment default is 10 old ReplicaSets.

Q3. What happens when revisionHistoryLimit is set to 2?

Kubernetes retains up to two old ReplicaSets, in addition to the current ReplicaSet.

Q4. Why does an old ReplicaSet show zero Pods?

Because the old ReplicaSet is retained for rollback, but its Pods are not required to run.

Q5. When does Kubernetes create a new Deployment revision?

When the Pod template changes, such as when the container image, environment variables or container configuration changes.

Q6. Does changing only the replica count create a new revision?

No. Changing only the number of replicas does not change the Pod template.

Q7. Which command displays Deployment revision history?

```
None
kubectl rollout history deployment/nginx-history
```

Q8. Which command displays ReplicaSets?

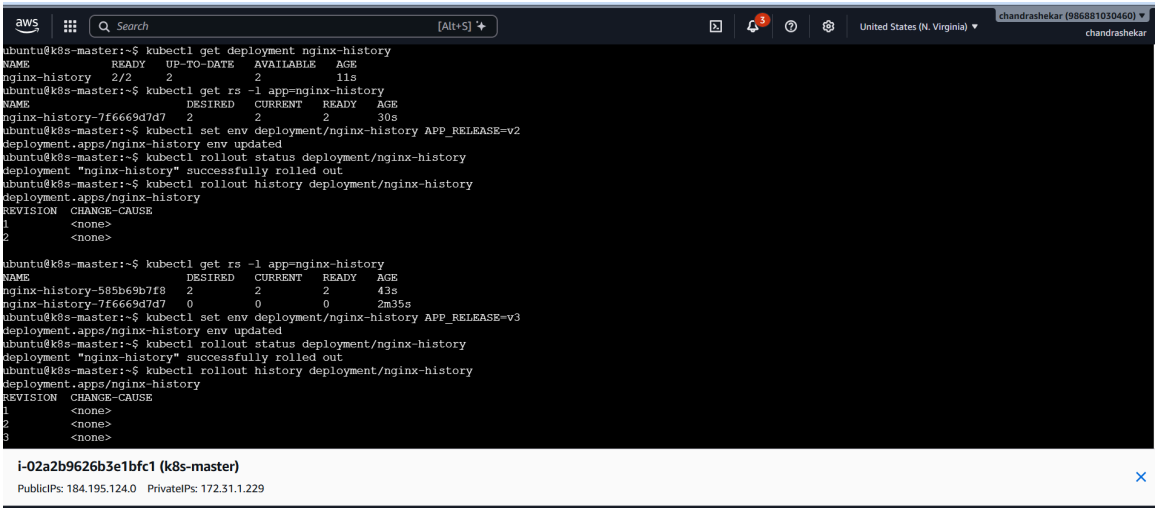
```
None
kubectl get rs -l app=nginx-history
```

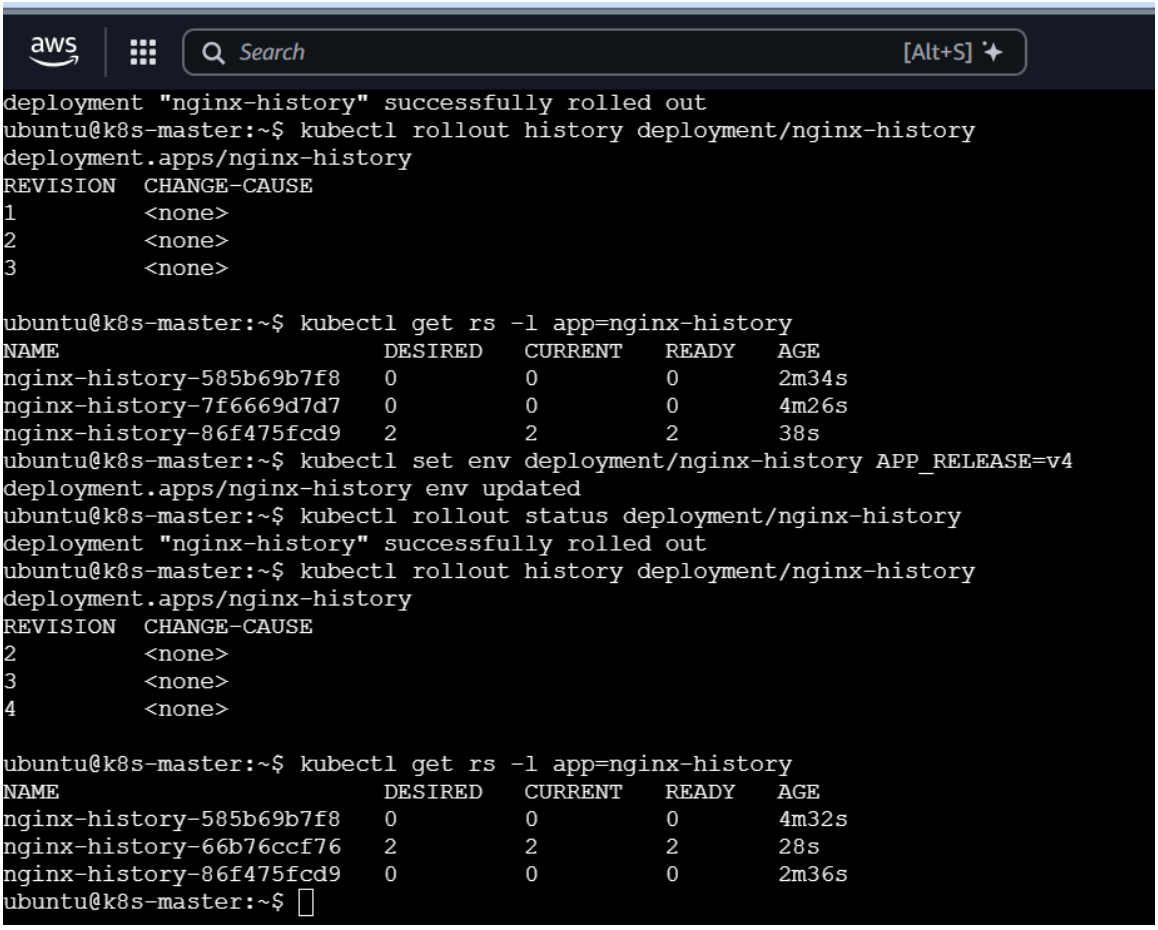
Q9. Can we roll back to an old revision that has been removed?

No. Once the required old ReplicaSet and revision information have been cleaned up, that revision is no longer available for a normal Deployment rollback.

Q10. Why do we use revisionHistoryLimit?

To control how many previous ReplicaSets Kubernetes retains for rollback and reduce unnecessary historical ReplicaSet storage.





K8S-04 Task 6: Completed successfully.

7.Pause a Deployment during an update, and then resume it.

1. Objective

The objective of this task is to pause a Kubernetes Deployment during an application update and then resume it to complete the rollout.

In this practical, we:

- Created an Nginx Deployment with three replicas.
- Paused the Deployment.
- Updated the application image from Nginx to Apache HTTPD while paused.
- Verified that the existing Nginx Pods were still running.
- Resumed the Deployment.
- Verified that the new HTTPD Pods were created successfully.

2. What is Pause in Kubernetes?

Pause means temporarily stopping the progress of a Deployment rollout.

When a Deployment is paused, Kubernetes does not continue replacing the old Pods with new Pods.

Real-world example:

Imagine a company is updating an online shopping application from version 1 to version 2.

During the update, the DevOps engineer wants to check some configuration changes before continuing.

The engineer can pause the Deployment, make the required changes, verify the configuration and then resume the Deployment.

3. What is Resume in Kubernetes?

Resume means allowing a previously paused Deployment to continue its rollout.

After resuming, Kubernetes starts progressing with the updated Pod template and replaces old Pods with new Pods.

4. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker nodes: k8s-worker-1 and k8s-worker-2
- Deployment name: nginx-pause
- Initial image: nginx
- Updated image: httpd
- Number of replicas: 3

5. Create Deployment YAML

File name: `nginx-pause.yaml`

```
None
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-pause
  labels:
    app: nginx-pause

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-pause

  strategy:
    type: RollingUpdate

  template:
    metadata:
      labels:
```

```
        app: nginx-pause
        tier: backend

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

6. Explanation of Important YAML Fields

- `apiVersion: apps/v1` – API version used for Deployment.
- `kind: Deployment` – Creates a Deployment resource.
- `metadata.name` – Name of the Deployment.
- `replicas: 3` – Maintains three Pods.
- `selector.matchLabels` – Identifies the Pods managed by the Deployment.
- `strategy.type: RollingUpdate` – Updates Pods gradually.
- `template` – Defines the Pod configuration.
- `image: nginx:latest` – Initially uses the Nginx image.
- `containerPort: 80` – Application listens on port 80.
- `tier: backend` – Label added to avoid interference from the earlier K8S-03 ReplicaSet using the `NotIn` selector.

7. Apply the Deployment

Command:

```
None
kubectl apply -f nginx-pause.yaml
```

Output:

```
None
deployment.apps/nginx-pause created
```

This confirms that the Deployment was created successfully.

8. Verify Deployment

Command:

```
None
kubectl get deployment nginx-pause
```

Output:

```
None
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-pause   3/3     3            3           15s
```

This confirms that all three replicas were ready and available.

9. Verify Initial Pods

Command:

```
None
kubectl get pods -l app=nginx-pause -o wide
```

Initial Pods:

Pod name	Worker node	Status
nginx-pause-6fb4d86b6f-jzqq8	worker-1	Running
nginx-pause-6fb4d86b6f-n9lsk	worker-2	Running

nginx-pause-6fb4d86b6f-s4t2p worker-2 Running

All three Nginx Pods were Running.

10. Pause the Deployment

Command:

```
None
kubectl rollout pause deployment/nginx-pause
```

Output:

```
None
deployment.apps/nginx-pause paused
```

This confirms that the Deployment was paused successfully.

11. Verify Paused Status

Command:

```
None
kubectl get deployment nginx-pause -o
jsonpath='{.spec.paused}{"\n"}'
```

Output:

```
None
true
```

This confirms that the Deployment was paused.

12. Update the Application Image While Paused

We changed the application image from Nginx to Apache HTTPD.

Command:

```
None
      kubectl set image deployment/nginx-pause
      nginx=httpd:latest
```

The Deployment's Pod template was updated.

However, because the Deployment was paused, Kubernetes did not start progressing with the new rollout.

13. Verify Existing Pods During Pause

Check paused status:

```
None
      kubectl get deployment nginx-pause -o
      jsonpath='{.spec.paused}{"\n"}'
```

Output:

```
None
      true
```

Check Pods:

```
None
      kubectl get pods -l app=nginx-pause
```

Output:

```
None
      NAME                           READY   STATUS
      RESTARTS   AGE
      nginx-pause-6fb4d86b6f-jzqq8   1/1     Running   0
      3m36s
```

```
nginx-pause-6fb4d86b6f-n9lsk   1/1     Running   0
3m36s
nginx-pause-6fb4d86b6f-s4t2p   1/1     Running   0
3m36s
```

This confirms that the original Nginx Pods continued running while the Deployment was paused.

14. Resume the Deployment

Command:

```
None
kubectl rollout resume deployment/nginx-pause
```

Output:

```
None
deployment.apps/nginx-pause resumed
```

This allows Kubernetes to continue the pending rollout.

15. Verify Rollout Status

Command:

```
None
kubectl rollout status deployment/nginx-pause
```

Output:

```
None
deployment "nginx-pause" successfully rolled out
```

This confirms that the updated Deployment completed successfully.

16. Verify New Pods

Command:

```
None
kubectl get pods -l app=nginx-pause -o wide
```

Final Pods:

Pod name	Worker node	Status
nginx-pause-684c598bfc-6mcpn	worker-2	Running
nginx-pause-684c598bfc-nxlr6	worker-1	Running
nginx-pause-684c598bfc-pm7cr	worker-2	Running

The new Pods have different names because Kubernetes created them using the updated Pod template.

All three new Pods were Running and Ready.

17. Verify Updated Application Image

Command:

```
None
kubectl get deployment nginx-pause -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"
}'
```

Output:

```
None
httpd:latest
```

This confirms that the updated application image is HTTPD.

18. Verify Deployment Is Resumed

Command:

```
None
kubectl get deployment nginx-pause -o
jsonpath='{.spec.paused}{"\n"}'
```

The command returned an empty output.

This is expected because the paused field is no longer set to true. An empty output here indicates that the Deployment is not paused.

19. Important Commands Used

Create Deployment:

```
None
kubectl apply -f nginx-pause.yaml
```

Check Deployment:

```
None
kubectl get deployment nginx-pause
```

Pause Deployment:

```
None
kubectl rollout pause deployment/nginx-pause
```

Check paused status:

```
None
kubectl get deployment nginx-pause -o
jsonpath='{.spec.paused}{"\n"}'
```

Update application image:

```
None
      kubectl set image deployment/nginx-pause
      nginx=httpd:latest
```

Resume Deployment:

```
None
      kubectl rollout resume deployment/nginx-pause
```

Check rollout status:

```
None
      kubectl rollout status deployment/nginx-pause
```

Check Pods:

```
None
      kubectl get pods -l app=nginx-pause -o wide
```

Check current image:

```
None
      kubectl get deployment nginx-pause -o
      jsonpath='{.spec.template.spec.containers[0].image}{"\n"
      }'
```

20. Final Result

Successfully created a Kubernetes Deployment with three Nginx replicas.

Paused the Deployment, updated its container image from Nginx to Apache HTTPD, and verified that the old Nginx Pods remained running during the pause.

Resumed the Deployment and confirmed that Kubernetes successfully replaced the old Pods with three new HTTPD Pods.

The rollout status was successful, and the final image verification showed `httpd:latest`.

21. Interview Questions and Answers

Q1. What is a paused Deployment?

A paused Deployment is a Deployment whose rollout progress is temporarily stopped.

Q2. Which command is used to pause a Deployment?

```
None
      kubectl rollout pause deployment/DEPLOYMENT_NAME
```

Q3. Which command is used to resume a Deployment?

```
None
      kubectl rollout resume deployment/DEPLOYMENT_NAME
```

Q4. Can we update an image while a Deployment is paused?

Yes. We can change the Deployment's Pod template while it is paused. The rollout does not progress until it is resumed.

Q5. What happens to existing Pods during a pause?

Existing Pods continue running unless something else causes them to stop or be removed.

Q6. Does pause delete existing Pods?

No. Pausing a Deployment does not delete its existing Pods.

Q7. What happens after resuming a Deployment?

Kubernetes continues the pending rollout and replaces old Pods according to the Deployment strategy.

Q8. How can we check whether a Deployment is paused?

```
None
```

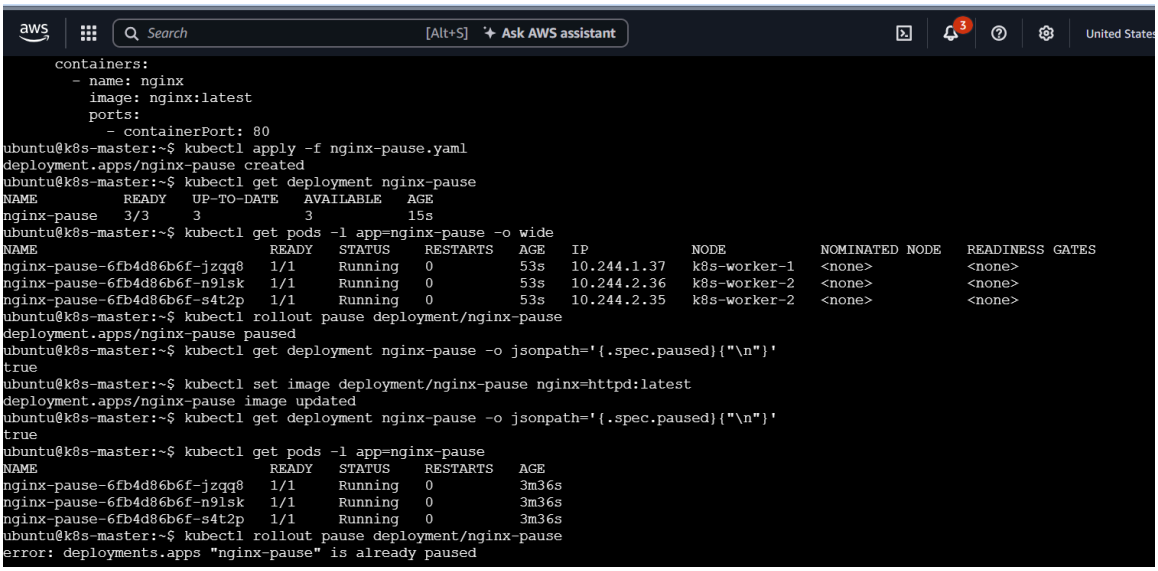
```
kubectl get deployment DEPLOYMENT_NAME -o
jsonpath='{.spec.paused}{"\n"}'
```

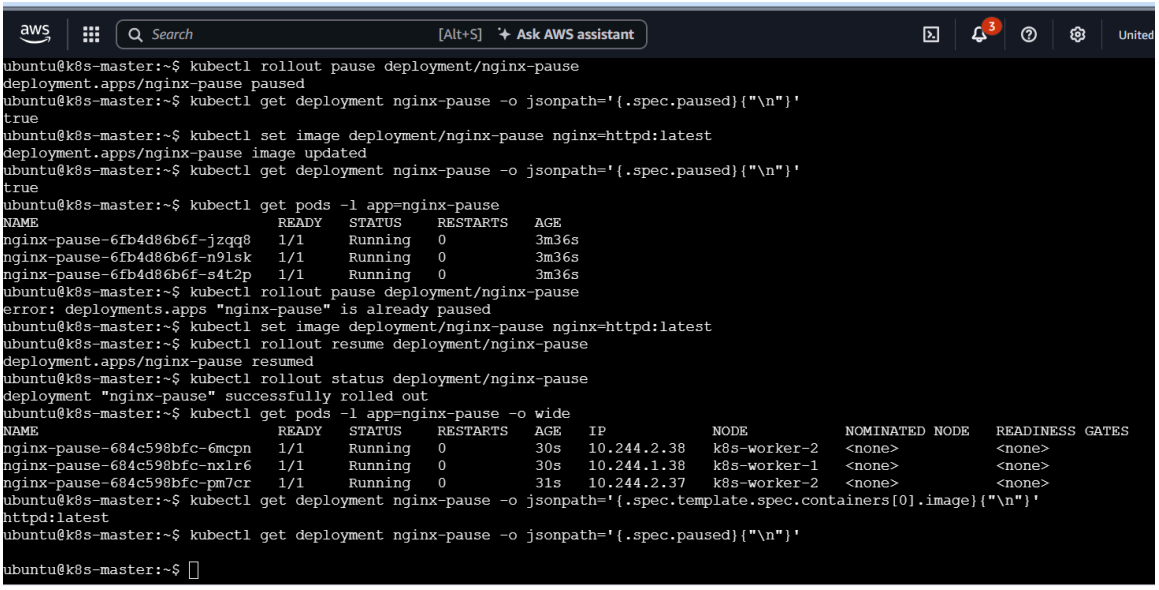
Q9. What happens if we try to pause an already paused Deployment?

Kubernetes reports that the Deployment is already paused.

Q10. Why do we use pause and resume in real projects?

They allow DevOps engineers to temporarily stop rollout progress, review or prepare changes, and continue the update when ready.





K8S-04 Task 7: Completed successfully.

8.Create a pod using resource requests for memory and CPU, and observe how the scheduler assigns it to a node.

1. Objective

The objective of this task is to create a Kubernetes Pod with CPU and memory resource requests and observe how Kubernetes Scheduler selects a suitable worker node.

In this practical, we:

- Created a Pod using YAML.
- Configured CPU and memory requests.
- Applied the YAML file.
- Checked the Pod status and assigned worker node.
- Verified scheduling details using `kubectl describe pod`.

2. What is Kubernetes Scheduler?

Kubernetes Scheduler is a control-plane component responsible for selecting a suitable worker node for a newly created Pod.

It checks the Pod's resource requests and the available schedulable capacity of nodes before assigning the Pod.

Real-world example:

Imagine a company has two worker nodes.

An application requires:

- CPU: 250m
- Memory: 256Mi

The Scheduler checks which worker node can satisfy these resource requests and selects a suitable node.

3. What are Resource Requests?

Resource requests specify the amount of CPU and memory needed by a container.

The Scheduler uses these values when deciding where to place the Pod.

In our practical:

Resource	Requested value
CPU	250m
Memory	256Mi

- `250m` means 250 millicores or 0.25 CPU core.
- `256Mi` means 256 mebibytes of memory.

Note: Resource requests are used for scheduling. They do not set the maximum resource usage. Limits are used to define maximum usage.

4. Environment

- Kubernetes cluster: AWS EC2
- Master node: k8s-master
- Worker node 1: k8s-worker-1
- Worker node 2: k8s-worker-2
- Pod name: resource-scheduler-pod
- Container image: nginx
- CPU request: 250m
- Memory request: 256Mi

5. Create Pod YAML

File name: `resource-scheduler-pod.yaml`

```
None
apiVersion: v1
kind: Pod
metadata:
  name: resource-scheduler-pod
  labels:
    app: resource-scheduler
    tier: backend

spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
```

6. Explanation of Important YAML Fields

- `apiVersion: v1` – API version used for Pod.
- `kind: Pod` – Creates a Pod resource.
- `metadata.name` – Name of the Pod.
- `labels` – Identifies and organizes the Pod.
- `containers` – Defines the container running inside the Pod.
- `name: nginx` – Container name.
- `image: nginx:latest` – Uses the Nginx container image.
- `containerPort: 80` – Application listens on port 80.
- `resources.requests` – Specifies required resources.
- `cpu: 250m` – Requests 0.25 CPU core.
- `memory: 256Mi` – Requests 256Mi memory.

The `tier: backend` label was added to avoid interference from the earlier K8S-03 ReplicaSet using a `NotIn` selector.

7. Apply the YAML File

Command:

```
None
kubectl apply -f resource-scheduler-pod.yaml
```

Output:

```
None
pod/resource-scheduler-pod created
```

This confirms that the Pod was created successfully.

8. Verify Pod Status and Worker Node

Command:

```
None
kubectl get pod resource-scheduler-pod -o wide
```

Output:

```
None
NAME                     READY   STATUS    RESTARTS   AGE   IP
NODE
resource-scheduler-pod   1/1     Running   0          13s
10.244.2.39   k8s-worker-2
```

This confirms:

- Pod was created successfully.
- Container is Running.
- Container is Ready.
- Pod IP is 10.244.2.39.

- Kubernetes Scheduler assigned the Pod to k8s-worker-2.

9. Verify Scheduling Details

Command:

```
None
kubectl describe pod resource-scheduler-pod
```

A. Verify assigned node

Output:

```
None
Node: k8s-worker-2/172.31.1.91
```

This confirms that the Pod was scheduled on worker node 2.

B. Verify resource requests

Output:

```
None
Requests:
  cpu:        250m
  memory:     256Mi
```

This confirms that the configured CPU and memory requests were applied.

C. Verify Pod status

Output:

```
None
Status:           Running
Ready:            True
Restart Count:    0
```

The container started successfully and did not restart.

D. Verify scheduling event

Output:

```
None
Events:
  Type    Reason     From               Message
  Normal  Scheduled  default-scheduler  Successfully assigned
default/resource-scheduler-pod to k8s-worker-2
```

This is important evidence for the task.

It confirms that the Kubernetes Scheduler selected k8s-worker-2 and assigned the Pod to that node.

The subsequent events also showed that the image was pulled and the container was created and started successfully.

10. Understand QoS Class – Burstable

The Pod description displayed:

```
None
QoS Class: Burstable
```

Kubernetes classifies Pods into three Quality of Service (QoS) classes:

QoS Class	Meaning
Guaranteed	Every container has equal CPU and memory requests and limits, with both specified
Burstable	The Pod has resource requests or limits but does not meet Guaranteed conditions
BestEffort	No CPU or memory requests or limits are specified

Our Pod has requests but no limits, so its QoS Class is Burstable.

11. Important Commands Used

Create the Pod:

```
None
kubectl apply -f resource-scheduler-pod.yaml
```

Check Pod status and worker node:

```
None
kubectl get pod resource-scheduler-pod -o wide
```

Describe Pod details:

```
None
kubectl describe pod resource-scheduler-pod
```

Check all cluster nodes:

```
None
kubectl get nodes -o wide
```

12. Final Result

Successfully created a Kubernetes Pod with the following resource requests:

- CPU: 250m
- Memory: 256Mi

Kubernetes Scheduler successfully assigned the Pod to `k8s-worker-2`.

Verified the assigned node, resource requests, Pod status and scheduling events using `kubectl describe pod`.

The Pod was Running and Ready with QoS Class Burstable.

13. Interview Questions and Answers

Q1. What is Kubernetes Scheduler?

It is a control-plane component that selects a suitable worker node for a Pod.

Q2. What is a resource request?

It specifies the CPU and memory needed by a container. The Scheduler considers these values when selecting a node.

Q3. What does 250m CPU mean?

It means 250 millicores, equal to 0.25 CPU core.

Q4. What does 256Mi memory mean?

It means 256 mebibytes of memory.

Q5. Which component assigns a Pod to a worker node?

Kubernetes Scheduler.

Q6. What happens if no worker node has enough available schedulable resources?

The Pod can remain in Pending status because the Scheduler cannot find a suitable node.

Q7. Which command shows the assigned worker node?

```
None
kubectl get pods -o wide
```

Q8. Which command shows the scheduling events?

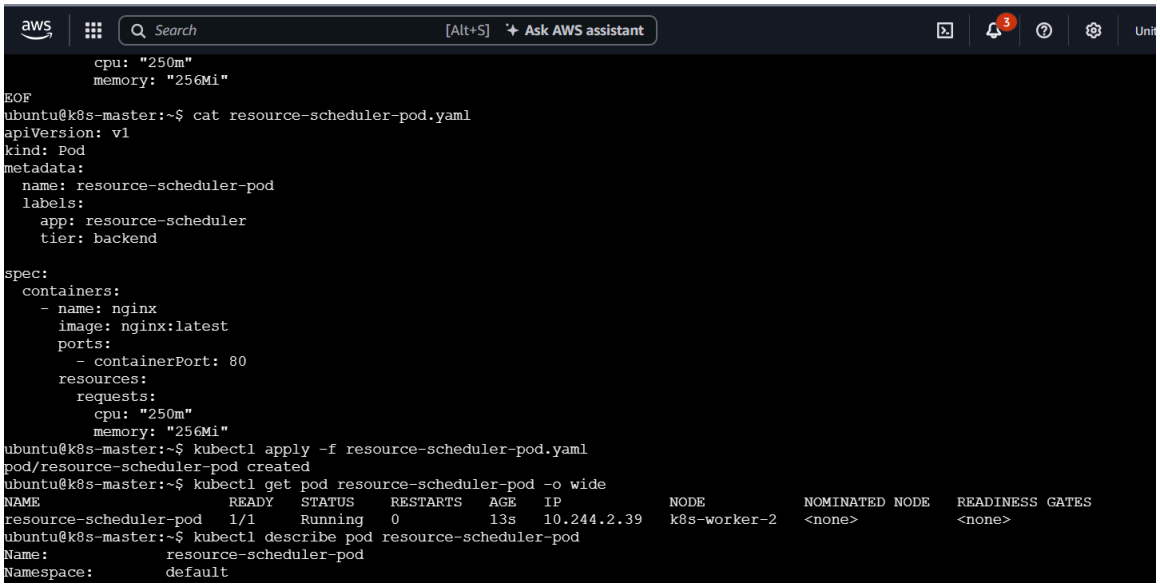
```
None
kubectl describe pod POD_NAME
```

Q9. What are the three QoS Classes?

Guaranteed, Burstable and BestEffort.

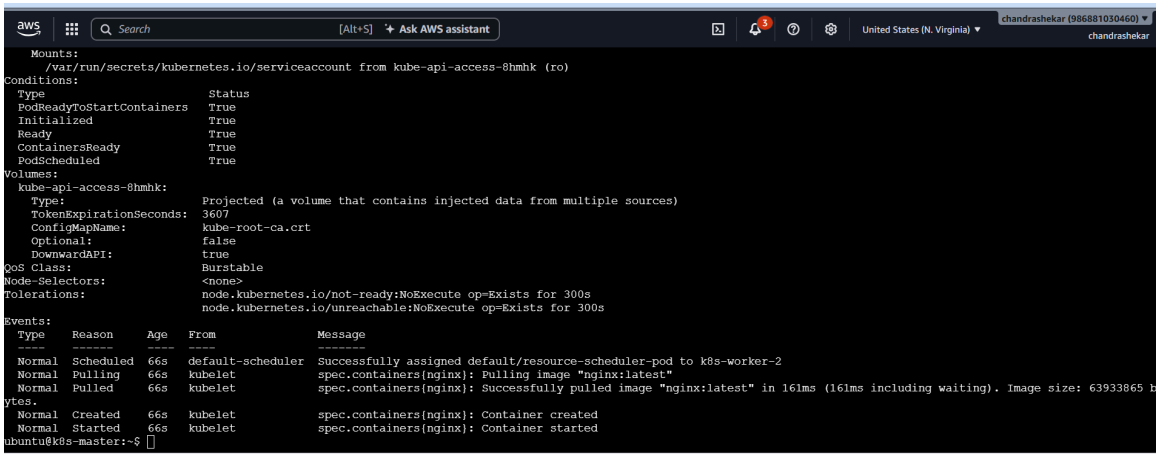
Q10. Does a resource request define the maximum resource usage?

No. Requests are used for scheduling. Resource limits define the maximum permitted usage.



```
          cpu: "250m"
          memory: "256Mi"
EOF
ubuntu@k8s-master:~$ cat resource-scheduler-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-scheduler-pod
  labels:
    app: resource-scheduler
    tier: backend

spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
ubuntu@k8s-master:~$ kubectl apply -f resource-scheduler-pod.yaml
pod/resource-scheduler-pod created
ubuntu@k8s-master:~$ kubectl get pod resource-scheduler-pod -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
resource-scheduler-pod   1/1     Running   0          13s   10.244.2.39   k8s-worker-2   <none>           <none>
ubuntu@k8s-master:~$ kubectl describe pod resource-scheduler-pod
Name:             resource-scheduler-pod
Namespace:        default
```



```
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-8hmhk (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-8hmhk:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  66s   default-scheduler  Successfully assigned default/resource-scheduler-pod to k8s-worker-2
  Normal  Pulling    66s   kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     66s   kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 161ms (161ms including waiting). Image size: 63933865 b
ytes.
  Normal  Created    66s   kubelet            spec.containers{nginx}: Container created
  Normal  Started    66s   kubelet            spec.containers{nginx}: Container started
ubuntu@k8s-master:~$
```

K8S-04 Task 8: Completed successfully.